

Personal Goals Tracker (“goalden”)

Submitted in partial fulfillment of the requirements of

Second Year

in

Computer Engineering

By

Student Vedant -- Roll No 28
Malviya

Student Dhruv -- Roll No 29
Mane

Student Kunal -- Roll No 35
Mokal

Student Aryan -- Roll No 41
Murkar

Guide

Prof. _____



Department of Computer Engineering
DATTA MEGHE COLLEGE OF ENGINEERING,

AIROLI, NAVI MUMBAI -400 708

University of Mumbai

(AY 2026-27)

CERTIFICATE

This is to certify that the Mini Project entitled **“Personal Goals Tracker (‘goalden’)** is a bonafide work of **Vedant Malviya (28), Dhruv Mane (29), Kunal Mokal (35) & Aryan Murkar (41)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **“Second year of Engineering”** in **“Computer Engineering”**.

(Prof. _____)

Guide

(Dr.A.P.Pande)

Head of Department

(Dr. S. D. Sawarkar)

Principal

Mini Project Approval

This Mini Project entitled “Personal Goals Tracker (‘goalden’)” by **Vedant Malviya (28), Dhruv Mane (29), Kunal Mokal (35) & Aryan Murkar (41)** is approved for the degree of “**Second year of Engineering**” in “**Computer Engineering**”.

Examiners

1.....
(Internal Examiner Name &Sign)

2.....
(External Examiner name &Sign)

Date:

Place:

Contents

Abstract

List of Abbreviations

List of Figures

1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Problem Statement and Objectives	2
2 Proposed System	4
2.1 Process Model (Waterfall / Spiral / Agile etc.)	4
2.2 Proposed Methodology / Architecture	4
2.3 Details of Hardware and Software	6
3 Implementation and Results	8
3.1 Results with Output	8
4 Conclusion and Future Work	11
References	12

Abstract

The "goalden" project is a personal goals tracker delivered as a web application with a static-first Astro frontend, Svelte 5 dashboard islands, and a Spring Boot REST API backed by PostgreSQL on Neon. Users register and sign in with short-lived access JWTs protected by rotating, server-side-hashed refresh tokens, then manage goals through a CRUD dashboard with per-status statistics. The deliverable covers authentication, goal CRUD, tenant-isolated ownership scoping, a Flyway-managed schema, and 26 offline integration tests, while the remaining product features - personal whiteboard, daily goal templates with weekend copying, subgoals, and progress charts - define the future roadmap.

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
BCrypt	Blowfish-based password hashing scheme
CI	Continuous Integration
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CORS	Cross-Origin Resource Sharing
H2	In-memory Java SQL database (used in tests)
HR	Human Resources
HTTP(S)	HyperText Transfer Protocol (Secure)
JPA	Java Persistence API
JWT	JSON Web Token
OKR	Objectives and Key Results
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface
URL	Uniform Resource Locator

List of Figures

Figure	Title	Page
2.1	Proposed System Architecture	5
3.1	Request Flow: Astro → Svelte Islands → Spring Boot → Neon	9

1 Introduction

1.1 Introduction

Goal setting is a well-established technique for personal development, yet most people track their goals in ad-hoc notes, spreadsheets, or paper diaries that give no feedback on progress and no structure for breaking large ambitions into achievable steps. The "goalden" project addresses this by providing a dedicated personal goals tracker that records goals with a title, notes, status, progress percentage and target date, and presents them through an interactive dashboard. The product is built as a modern full-stack web application: an Astro static-first frontend with Svelte 5 islands, a Spring Boot REST API on Java 21, and a serverless PostgreSQL database hosted on Neon, with the schema owned by Flyway migrations.

The delivered vertical slice comprises a landing page, sign-in and registration screens, and a goal dashboard supporting full CRUD, filtering, deadline warnings and per-status statistics, all protected by a hardened token-based authentication scheme with refresh-token rotation and reuse detection.

1.2 Motivation

Large goals are difficult to act on ("eat the elephant"), daily intentions are easily lost between weekends, and progress that is never visualised is quickly abandoned. The motivation for this project is threefold:

- Product motivation: build a personal whiteboard in the spirit of Excalidraw, reusable daily goal templates copied across the weekend, monthly-and-longer goals carrying explicit deadlines, subgoals that divide large ambitions, and charts with animations to make progress visible.
- Engineering motivation: practice a production-grade architecture - static-first rendering with selective hydration, a layered REST API, migration-driven schema

evolution, and security mechanisms such as token rotation, hashing at rest and strict ownership scoping.

- Academic motivation: satisfy the mini-project requirement of the Second Year Computer Engineering programme at Datta Meghe College of Engineering by integrating frontend, backend, database and DevOps concerns in one coherent deliverable.

1.3 Problem Statement and Objectives

Problem statement: Individuals lack a single, secure, always-available tool that combines long-term and daily goal tracking with visual progress feedback, so goals are recorded inconsistently and progress goes unmeasured.

Objectives:

1. Provide secure user registration, login, logout and session refresh using short-lived JWT access tokens with rotating, hashed refresh tokens.
2. Deliver a goal CRUD dashboard with per-status statistics (PENDING, IN_PROGRESS, COMPLETED, ARCHIVED), completion rate, average progress and deadline sorting.
3. Enforce tenant isolation so a user can never read, modify or confirm the existence of another user's goals (404 rather than 403).
4. Keep the schema under versioned Flyway migrations validated by Hibernate, and cover the API with offline integration tests using H2 in PostgreSQL compatibility mode.
5. Plan (future increments) the whiteboard, daily templates with weekend copying, subgoals and charts, plus production readiness items such as email verification, password reset and rate limiting.

2 Proposed System

2.1 Process Model (Agile)

The project follows an iterative Agile process. Work proceeded in small, vertical increments - each one taking a feature from UI through API to persistence with tests - rather than the sequential phases of a Waterfall model. Git history records seven commits, including the feature increments "Add Landing Page", "USER Authentication" and "Implement Goals" made over 28–29 September 2026. This incremental approach let authentication, goal CRUD and statistics ship early and be tested independently, while the larger roadmap items (whiteboard, templates, subgoals, charts) remain prioritised backlog entries.

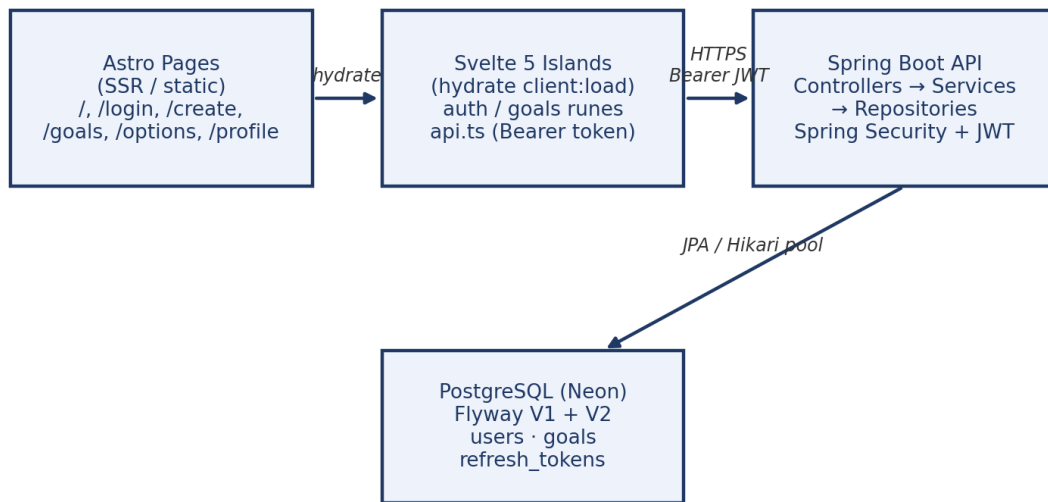
2.2 Proposed Methodology / Architecture

The system follows a three-tier client-server architecture with static-first delivery:

- Presentation tier - Astro 7.3.5 serves file-based routes (`/`, `/login`, `/create`, `/goals`, `/options`, `/profile`); only the screens that need state hydrate as Svelte 5 islands, keeping the marketing page light.
- Application tier - Spring Boot 3.5.16 on Java 21 exposes a REST API organised by feature (auth, goal, user, health, config, common) in strict controller → service → repository layers. Entities are never serialised directly; they map to `*Response` records so sensitive columns such as `password_hash` cannot leak.
- Data tier - PostgreSQL on Neon is reached through Spring Data JPA/Hibernate (`ddl-auto=validate`) with a Hikari pool of 10, and the schema is created exclusively by Flyway migrations V1 (users, goals) and V2 (refresh_tokens).

The security methodology layers BCrypt password hashing, HS256 JWT access tokens (default PT12H), refresh tokens stored only as SHA-256 hashes with rotation and theft detection (presenting an already-rotated token revokes all sessions), role-gated admin endpoints (ROLE_ADMIN on `/api/users/**`), stateless sessions with CSRF disabled, and an ownership model that resolves the user from the token - never from the request body.

Figure 2.1: Proposed System Architecture



2.3 Details of Hardware and Software

Layer	Technology / Version	Declared in
Frontend	Astro 7.3.5, @astrojs/svelte 9.0.1, Svelte 5.57.1 (runes), TypeScript 6.0.3, Node >= 22.12	frontend/package.json
Styling and icons	Tailwind CSS 4.3.3 via @tailwindcss/vite with a custom @theme; astro-icon + akar-icons	global.css, package.json
Backend	Spring Boot 3.5.16 on Java 21, Maven wrapper (mvnw), jar build	java-backend/pom.xml
Persistence	PostgreSQL via Neon; Spring Data JPA + Hibernate (ddl-auto=validate); Hikari pool of 10	application.yml
Schema	Flyway migrations V1 (users, goals) and V2 (refresh_tokens)	src/main/resources/db/migration
Security	Spring Security, BCrypt, jjwt 0.12.7 (HS256) bearer tokens	pom.xml, SecurityConfig
Configuration	.env imported by Spring:	.env.example, application.yml

	DATABASE_URL, JWT_SECRET, CORS_ALLOWED_ORIGINS	
Testing	JUnit 5, MockMvc, AssertJ; H2 in PostgreSQL compatibility mode	src/test
Hardware (development)	Standard developer workstation; serverless Neon database (scale- to-zero, pooled connection)	-

3 Implementation and Results

3.1 Results with Output

The API implements the following endpoints and behaviours:

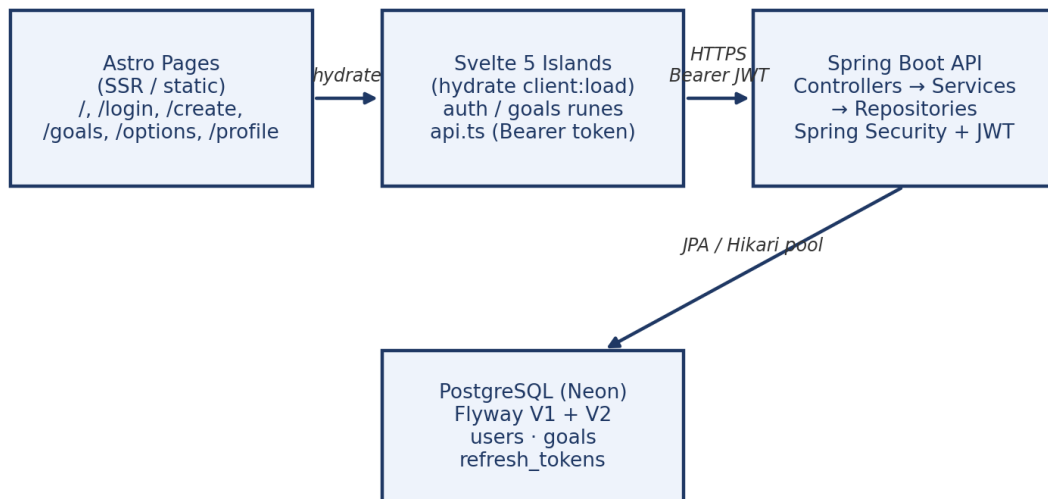
Method and path	Access	Behaviour
GET /api/health	public	Liveness check
POST /api/auth/register, /login, /refresh, /logout	public	Token lifecycle: issue, rotate, revoke
GET /api/auth/me	signed in	Current user resolved from the token
GET, POST /api/goals	signed in	List newest-first / create with 201 + Location header
GET, PUT, DELETE /api/goals/{id}	owner	Fetch, partial update, 204 delete
GET /api/goals/stats	owner	Counts per status (PENDING, IN_PROGRESS, COMPLETED, ARCHIVED)
GET /api/users; GET, PUT, DELETE /api/users/{id}	ROLE_ADMIN	List, fetch and manage users

Key implementation results:

- Authentication: registration and login issue an access/refresh token pair; refresh rotates the token on every use, and reuse is treated as theft, revoking all sessions for that user. Password change revokes refresh tokens; startup fails fast when JWT_SECRET is missing or shorter than 32 characters.
- Goal management: create returns 201 with a Location header; listing is newest-first; the stats endpoint reports counts per status for the signed-in user only.
- Validation contract: every failure returns one ApiError JSON body (timestamp, status, message, path, fieldErrors) - 400 validation, 401 missing/invalid token, 403 wrong role, 404 missing or not-yours, 409 duplicate email.

- Frontend dashboard: a summary strip (total, still open, completed, completion %) above an average-progress bar; a create form with a 0–100 progress slider; a deadline timeline; banners for goals due within 45 days or overdue; status filter chips; and a card grid whose inline slider and status select write directly back to the API, with two-click delete and rollback on rejection.
- Test output: 26 tests in three classes run offline on H2 in PostgreSQL mode - registration, login, goal CRUD, ownership scoping between users, the admin gate, validation failures, refresh rotation, reuse revocation, logout, and CORS preflight from the Astro origin.

Figure 3.1: Request Flow — Astro renders, Svelte islands hydrate and call the Spring Boot API over CORS-checked HTTPS; the API authorises via JWT and persists to Neon through Flyway-owned tables.



Real-world applicability: the session-handling pattern suits consumer sign-in flows and revocable third-party credentials; ownership scoping with 404-not-403 is the isolation model required for finance, health or HR data; scale-to-zero Postgres suits hobby and staging deployments; migration files that are never edited after deployment keep schema changes reviewable; and the goal domain model generalises to OKR tools, coaching or therapy homework trackers, training and rehabilitation programs, and course or compliance completion.

4 Conclusion and Future Work

The project delivers a coherent vertical slice: an Astro landing page and auth screens, a Svelte 5 goal dashboard, and a Spring Boot API whose token rotation, ownership scoping, validation contract and Flyway-managed schema are covered by 26 offline tests. The goals table lacks a `parent_id` column, so subgoals require a new V3 migration - V1 may still be edited only because it has never been deployed. Documentation drift exists (the backend README still lists refresh tokens as missing), and `/options` and `/profile` remain empty layout shells.

Future work:

- Personal whiteboard in the spirit of Excalidraw.
- Reusable daily goals template copied automatically across the weekend.
- Subgoals under a parent goal (requires Flyway V3 migration adding `parent_id`).
- Pie charts and animations for progress visualisation.
- Content for the `/options` (preferences) and `/profile` (account detail) pages.
- Production readiness: email verification, password reset flow, rate limiting on `/api/auth/*`, and strict (non-partial) PUT semantics.
- Correct the backend README to reflect refresh-token rotation, reuse detection and the V2 migration.

References

[1] Astro documentation - <https://docs.astro.build>

[2] Svelte 5 documentation - <https://svelte.dev/docs>

[3] Spring Boot reference documentation - <https://docs.spring.io/spring-boot/docs/current/reference/html/>

[4] Spring Security reference - <https://docs.spring.io/spring-security/reference/>

[5] Flyway documentation - <https://documentation.red-gate.com/flyway>

[6] Neon serverless PostgreSQL documentation - <https://neon.tech/docs>

[7] JSON Web Tokens (RFC 7519) - <https://www.rfc-editor.org/rfc/rfc7519>